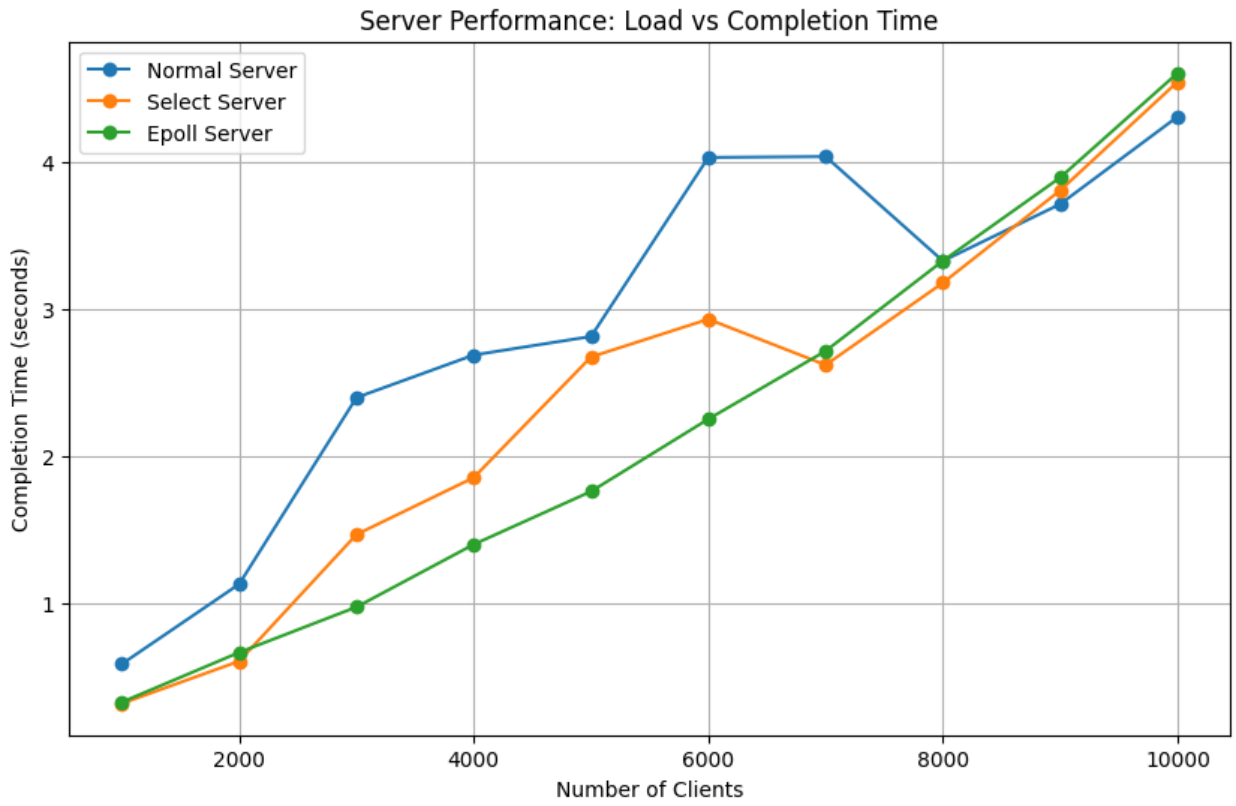
Kserver – 14 – listen 10

Kserver_select – 1270 – listen 10

Report: 4a

4. Plot : Performance Comparison of server.c, server_select.c and server_epoll.c



● Normal Server: This server consistently shows the worst performance, with completion times increasing sharply as the load grows. But, By 10000 clients, it takes significantly lesser time than both Select and Epoll, indicating that it struggles to manage large numbers of connections.

● Select Server: The Select server shows competitive performance, particularly for lower client counts (up to 2000 clients). It scales well with load, and in some cases (e.g., around 7000 clients), it performs slightly better than the Epoll server.

● Epoll Server: The Epoll server also scales well, showing nearly linear

performance similar to the Select server. While it generally performs better than Normal, its performance is closely matched by Select, with Epoll taking a slight lead at higher client loads (10000 clients).

Select() and Epoll() similarities

The reason why select and epoll servers give similar curves might lie in the backlog size. I experimented with a backlog size of 10 fds. Select linearly checks the whole list of fds for active connections while epoll monitors only the active connections. But with a backlog size of 10 the overhead of linearly going over the fd list is not tolling the select performance significantly thus resulting in an almost identical performance with epoll.

Better performance of the simple server

The reason why the simple server is performing better than select and epoll might be the overhead select and epoll are accumulating due to their polling mechanisms given the client size and not-so-rigorous work load.

5. Backlog = 10

kserver.c (Normal Server):

● Connection failed with 14 clients.

● This indicates that the Normal Server could not handle more than 14 clients with the given backlog size of 10. The server's queue filled up quickly, leading to connection refusal for subsequent clients. This suggests that the Normal Server implementation struggles with handling even moderate loads and may not effectively utilize the backlog limit.

kserver_select.c (Select Server):

● Connection failed with 1270 clients.

● The Select Server handles a significantly higher load compared to the Normal Server before reaching the connection refusal point. This suggests that the select() system call is more efficient in managing the server's connection handling, allowing it to scale to higher client numbers without overwhelming the connection queue too quickly.

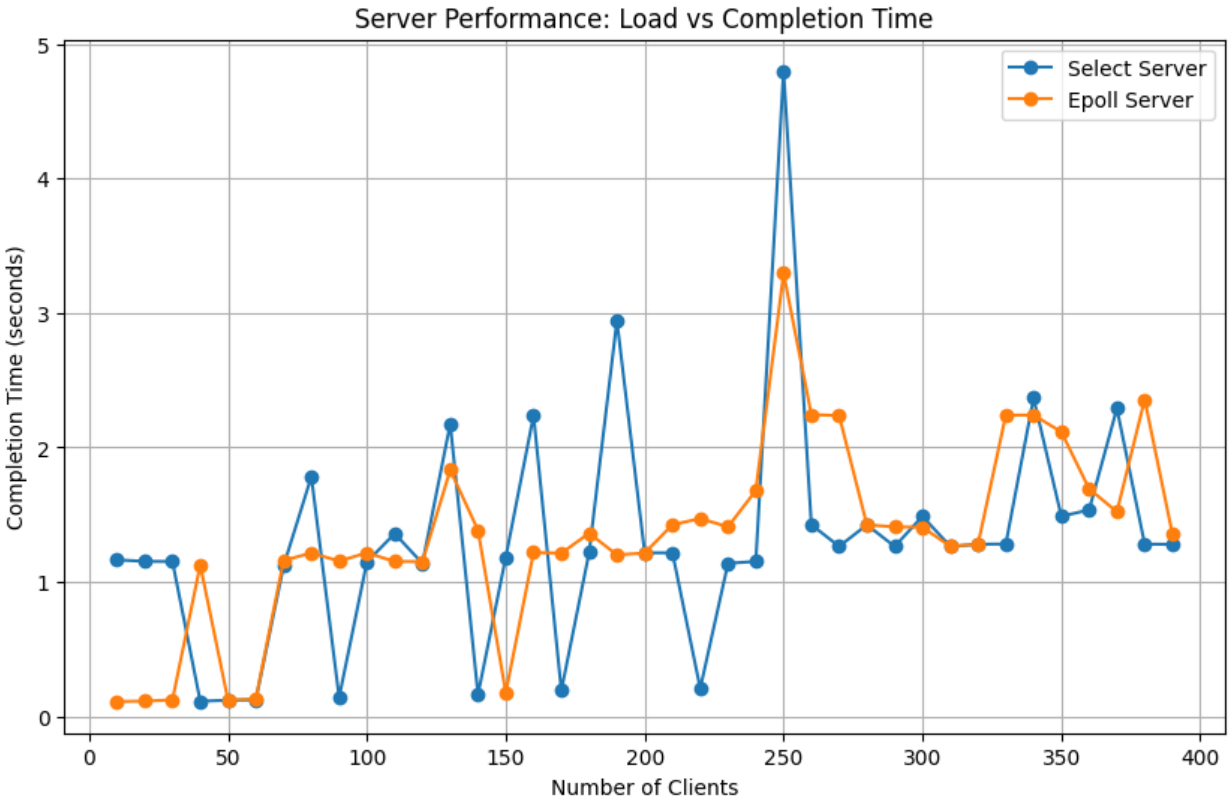
kserver_epoll.c (Epoll Server):

● Connection failed with 1918 clients.

● The Epoll Server outperforms both the Normal and Select servers by a substantial margin. It can manage up to 1918 clients before experiencing connection refusal,

demonstrating that the epoll() system call mechanism is highly scalable and can efficiently handle a large number of connections without saturating the server's pending connection queue as quickly as the other implementations.

6. Plot: Comparison between select() and epoll() performance under stress



- The *Select Server* (in blue) demonstrates more significant fluctuations in completion time as the number of clients increases. Peaks are noticeable at around 150 and 250 clients, indicating occasional performance degradation.
- In contrast, the *Epoll Server* (in orange) shows relatively stable completion times across different client loads, with minor fluctuations.

- As the number of clients increases from around 250 to 300, the *Select Server* experiences a sharp peak in completion time. The *Epoll Server*, however, manages a steadier completion time during this period.
- At higher loads (beyond 300 clients), both servers show some fluctuations, but the *Epoll Server* still outperforms the *Select Server* in stability.

- The *Epoll Server* is likely using a more efficient handling mechanism, allowing it to better cope with increasing loads. This results in fewer spikes in completion time, suggesting better scalability compared to the *Select Server*.
- The *Select Server* seems to experience issues under higher client loads, which results in periodic spikes in completion time. This could indicate resource contention or less efficient client handling as the number of concurrent clients increases.